

期中作业实验报告

姓名：庄子硕 学号：2350998 日期：2025 年 5 月 4 日

- # 实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
- # 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
- # 实验报告要求在文字简洁的同时将内容表示清楚
- # 报告内不要大段贴代码，尽量控制在 20 页以内

1. 编程语言和环境

题 1、2、3 都有 Mini-conda 下 python 3.8 实现，使用 d2l 及其依赖库；题 3 还提供了 cpp 实现。

2. 实验内容

2.1 题目一：排序算法综合应用

2.1.1 问题描述

假设高考科目包括语文、数学、英语这三门，每门课的分数为非负整数，满分为 150 分。现在要对某个地区的考生，根据他们的得分情况进行排名，按总分从高到低进行排序。总分相同的情况下按照先语文后数学再英语的顺序进行排名。

例：

排名	考号	姓名	总分	语文	数学	英语
1	XXXX	XXX	450	150	150	150
2			449	149	150	150
3			448	150	150	148
4			448	150	149	149
4			448	150	149	149
6			448	150	148	150
7			448	149	150	149
8			445	150	150	145

2.1.2 基本要求

请设计合适的算法，完成上述任务。

输入：考生数为 n ，考生的成绩由程序随机生成，并写入文本文件。成绩尽可能符合正态分布。

输出：符合要求的考生分数排名表，以文本文件的形式记录

分段设置 n 的范围，采用你认为最合适的排序算法。

- (1) n 较小 ($1-50,000$)，待排元素能一次放进内存；并给出排序时间随 n 变化情况（用曲线图表示）
- (2) n 较大，输入正整数 m ， $m < n$ ，求所有 n 位考生中排名前 m 位的考生

2.1.3 数据结构设计

```
class student: # 定义 student 类型
    def __init__(self, num, name, total_score, math_score, english_score,
chinese_score):
        self.num = num
        self.name = name
        self.total_score = total_score
        self.math_score = math_score
        self.english_score = english_score
        self.chinese_score = chinese_score

    def __lt__(self, other): # 定义小于关系, 方便后续运算符重载
        if(self.total_score != other.total_score):
            return self.total_score > other.total_score
        elif(self.chinese_score != other.chinese_score):
            return self.chinese_score > other.chinese_score
        elif(self.math_score != other.math_score):
            return self.math_score > other.math_score
        elif(self.english_score != other.english_score):
            return self.english_score > other.english_score
        else:
            return self.num < other.num
```

2.1.4 功能说明（算法思想、函数、类）

(1) 测试快速排序和冒泡排序, 前者采用分治思想复杂度低; 后者逐个排序, 复杂度高。

```
def partition(students, low, high): # quick-sort 的划分函数
    pivot = students[high]
    i = low - 1
    for j in range(low, high):
        if students[j] < pivot:
            i += 1
            students[i], students[j] = students[j], students[i]
    students[i + 1], students[high] = students[high], students[i + 1]
    return i + 1

def quick_sort(students, low, high): # 快速排序
    if low < high:
        pi = partition(students, low, high)
        quick_sort(students, low, pi - 1)
        quick_sort(students, pi + 1, high)

def bubble_sort(students): # 冒泡排序
    n = len(students)
```

```

for i in range(n):
    for j in range(0, n-i-1):
        if students[j] > students[j+1]:
            students[j], students[j+1] = students[j+1], students[j]
(2) 外部归并排序：找最大的 m 名学生，只需维护一个 m 大小的小根堆
def find_top_m_students(filename, m):
    heap = []
    with open(filename, "r") as f:
        for line in f:
            num, name, total, math, eng, chi = line.strip().split()
            stu = Student(int(num), name, int(total), int(math), int(eng),
int(chi))
            if len(heap) < m:
                heapq.heappush(heap, stu)
            else:
                if stu > heap[0]:
                    heapq.heapreplace(heap, stu)
    # 按降序输出
    return sorted(heap, reverse=True)

```

2.1.5 调试分析（算法分析和输入输出）

(1) 选择一系列 N 值，测试排序时间并可视化。可见快排明显优于冒泡。

```

N=[50,250,500,2500,5000,25000,50000] # Number of students
quick_sort_time = []
bubble_sort_time = []

```

```

for n in N:
    # Generate random student data
    with open("students.txt", "w") as f:
        for i in range(1, n+1):
            num = i
            name = f"Student{i}"
            total_score = 0
            math_score = 0
            english_score = 0
            chinese_score = 0

            # Generate random scores for each subject
            math_score = random.randint(0, 150)
            english_score = random.randint(0, 150)
            chinese_score = random.randint(0, 150)

            # Calculate total score

```

```

        total_score = math_score + english_score + chinese_score

        # Write to file in the specified format
        f.write(f"{num} {name} {total_score} {math_score} {english_score}
{chinese_score}\n")

students = []

with open("students.txt", "r") as f:
    lines = f.readlines()
    for line in lines:
        parts = line.strip().split()
        num = int(parts[0])
        name = parts[1]
        total_score = int(parts[2])
        math_score = int(parts[3])
        english_score = int(parts[4])
        chinese_score = int(parts[5])
        students.append(student(num, name, total_score, math_score,
english_score, chinese_score))

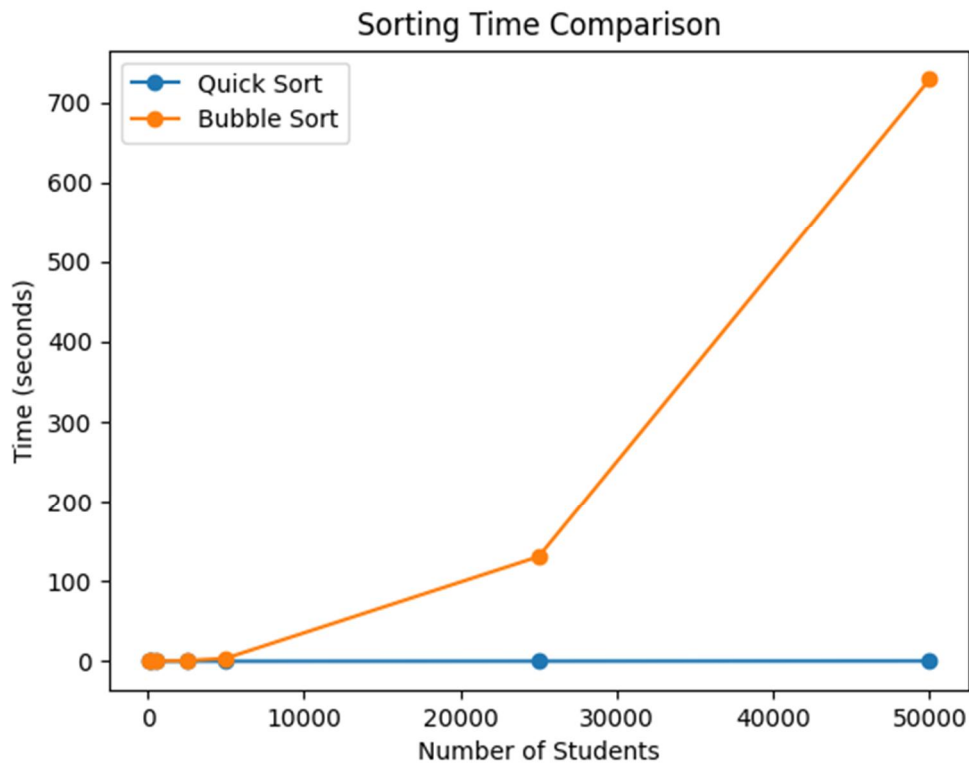
quick_sort_start_time = time.time()
sorted_students = quick_sort(students, 0, len(students) - 1)
quick_sort_end_time = time.time()
quick_sort_time.append(quick_sort_end_time - quick_sort_start_time)

bubble_sort_start_time = time.time()
bubble_sort(students)
bubble_sort_end_time = time.time()
bubble_sort_time.append(bubble_sort_end_time - bubble_sort_start_time)

# 存储排序后 student
# with open("sorted_students.txt",w) as out:
#     for student in students:
#         out.write(f"{student.num} {student.name} {student.total_score}
{student.math_score} {student.english_score} {student.chinese_score}\n")

```

运行结果如下：



(2) 归并排序内容输出到文件

```
top_students=find_top_m_students("students.txt", m)
with open("top_students.txt", "w") as f:
    for student in top_students:
        f.write(f"{student.num} {student.name} {student.total_score}
{student.math_score} {student.english_score} {student.chinese_score}\n")
```

部分输出如下:

```
108952 Student108952 446 148 150 148
59521 Student59521 446 149 150 147
68828 Student68828 445 150 148 147
54341 Student54341 445 148 150 147
124676 Student124676 444 147 147 150
15775 Student15775 444 147 147 150
61060 Student61060 444 148 148 148
69649 Student69649 444 146 150 148
73119 Student73119 442 150 143 149
94164 Student94164 442 149 146 147
34125 Student34125 441 150 142 149
113095 Student113095 441 147 145 149
134505 Student134505 441 143 149 149
54757 Student54757 441 143 149 149
134300 Student134300 441 146 149 146
126050 Student126050 440 146 146 148
54502 Student54502 440 142 150 148
```

2.2 题目二：分治算法

2.2.1 问题描述

给定线性序集中 n 个元素和一个整数 k , $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素, (这里给定的线性集是无序的)。

2.2.2 基本要求

下面三种是可行的方法:

(1) 基于堆的选择: 不需要对全部 n 个元素排序, 只需要维护 k 个元素的最大堆, 即用容量为 k 的最大堆存储最小的 k 个数, 总费时 $O(k + (n-k) * \log k)$

(2) 随机划分线性选择 (教材上的 RandomizedSelect): 在最坏的情况下时间复杂度为 $O(n^2)$, 平均情况下期望时间复杂度为 $O(n)$ 。

(3) 利用中位数的线性时间选择: 选择中位数的中位数作为划分的基准, 在最坏情况下时间复杂度为 $O(n)$ 。

请给出以上三种方法的算法描述, 用你熟悉的编程语言实现上述三种方法。并通过实际用例测试, 绘出三种算法的运行时间随 k 和 n 变化情况的对比图 (表), 特别是 n 较大时方法 (2) 和 (3) 的对比。

2.2.3 数据结构设计

大根堆, 维护最小的 k 个元素

```
class k_min_heap:
```

```
    def __init__(self, k):
```

```
        self.k = k
```

```
        self.heap = []
```

```
    def insert(self, value):
```

```
        if len(self.heap) < self.k:
```

```
            self.heap.append(value)
```

```
            self._heapify_up(len(self.heap) - 1)
```

```
        elif value < self.heap[0]:
```

```
            self.heap[0] = value
```

```
            self._heapify_down(0)
```

```
    def _heapify_up(self, index):
```

```
        while index > 0:
```

```
            parent = (index - 1) // 2
```

```
            if self.heap[index] > self.heap[parent]: # 大顶堆
```

```
                self.heap[index], self.heap[parent] =
```

```
self.heap[parent], self.heap[index]
```

```
                index = parent
```

```
            else:
```

```
                break
```

```

def _heapify_down(self, index):
    size = len(self.heap)
    while True:
        largest = index
        left = 2 * index + 1
        right = 2 * index + 2
        if left < size and self.heap[left] >
self.heap[largest]:
            largest = left
        if right < size and self.heap[right] >
self.heap[largest]:
            largest = right
        if largest != index:
            self.heap[index], self.heap[largest] =
self.heap[largest], self.heap[index]
            index = largest
        else:
            break

def get_k_min(self, array):
    for i in range(len(array)):
        self.insert(array[i])
    return self.heap[0]

```

2.2.4 功能说明（算法思想、函数、类）

算法复杂度分析见问题描述

```

# 优化的快排：随机选择划分项
def randomized_partition(arr, low, high):
    # 随机选择主元并与最后一个元素交换
    pivot_idx = random.randint(low, high)
    arr[pivot_idx], arr[high] = arr[high], arr[pivot_idx]
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i + 1

def randomized_select(arr, low, high, k):
    if low == high:
        return arr[low]

```

```

    pivot_idx = randomized_partition(arr, low, high)
    rank = pivot_idx - low + 1
    if k == rank:
        return arr[pivot_idx]
    elif k < rank:
        return randomized_select(arr, low, pivot_idx-1, k)
    else:
        return randomized_select(arr, pivot_idx+1, high, k-rank)

# 优化的快排：中位数的中位数划分
def median_of_medians_select(arr, k):
    if len(arr) <= 5:
        return sorted(arr)[k-1]

    groups = [arr[i:i+5] for i in range(0, len(arr), 5)]
    medians = [sorted(group)[len(group)//2] for group in
groups]
    pivot = median_of_medians_select(medians, len(medians)//2 +
1)

    left = [x for x in arr if x < pivot]
    right = [x for x in arr if x > pivot]
    pivots = [x for x in arr if x == pivot]

    if k <= len(left):
        return median_of_medians_select(left, k)
    elif k <= len(left) + len(pivots):
        return pivot
    else:
        return median_of_medians_select(right, k - len(left) -
len(pivots))

```

2.2.5 调试分析（算法分析和输入输出）

```

# 测试代码
array=[]
time_cost=[]

N=[100,1000,10000,100000]
for n in N:
    K=[n//1000,n//500,n//200,n//100,n//50,n//10,n//5,n//3,n//2]

    for i in range(0,len(K)):

```



```

        if K[i] == 0:
            K[i] = 1

for k in K:
    array.clear()
    # print(f"随机选择第{k}小元素\n")
    for i in range(1, n+1):
        array.append(random.randint(0, 1000))

    # print("原始数组:", array)

    o1=k_min_heap(k)
    start1=time.time()
    elem1=o1.get_k_min(array)
    end1=time.time()
    time_cost.append(end1-start1)
    # print(f"基于堆选择第{k}小元素: ",elem1)

    start2=time.time()
    elem2=randomized_select(array, 0, len(array)-1, k)
    end2=time.time()
    time_cost.append(end2-start2)
    # print(f"基于随机选择第{k}小元素: ",elem2)

    start3=time.time()
    elem3=median_of_medians_select(array, k)
    end3=time.time()
    time_cost.append(end3-start3)
    # print(f"基于中位数选择第{k}小元素: ",elem3)

time_cost = np.array(time_cost).reshape(len(N), len(K)*3)

fig, axes = plt.subplots(1, 4, figsize=(18, 5)) # 一行三列

for i in range(len(N)):
    n=N[i]
    K=[n//1000,n//500,n//200,n//100,n//50,n//10,n//5,n//3,n//2]
    heap_time = time_cost[i][0::3]
    rand_time = time_cost[i][1::3]
    mom_time = time_cost[i][2::3]
    axes[i].plot(K, heap_time, label="Heap",marker='o')
    axes[i].plot(K, rand_time, label="Random",marker='o')
    axes[i].plot(K, mom_time, label="Median",marker='o')
    axes[i].set_xlabel('K')

```

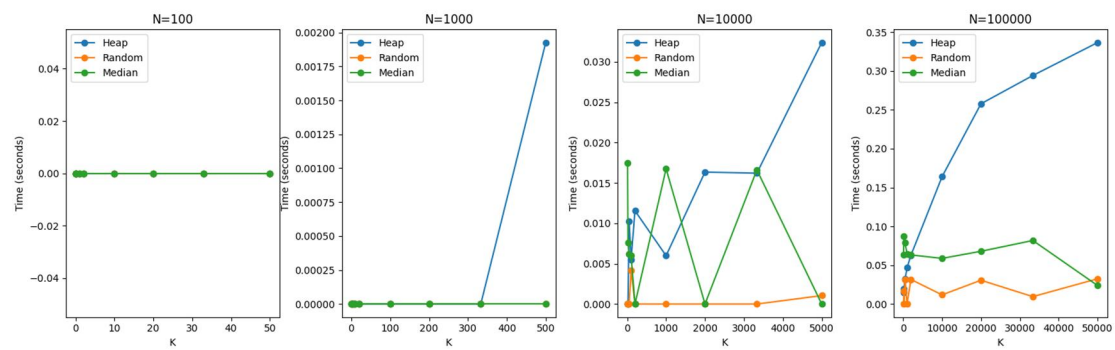
```

axes[i].set_ylabel('Time (seconds)')
axes[i].set_title(f'N={N[i]}')
axes[i].legend(loc='upper left')

plt.tight_layout()
plt.show()

```

运行结果如下：



可见，在 n 、 k 比较大时堆排序与优化的快排差距较大；另外，在 n 固定，增大 k 时，两个快排优化方法的复杂度没有明显变化，而堆排序耗时明显变大。

2.3 题目三：动态规划

2.3.1 问题描述

Uncle Mike has passed away and in his will has left a collection of valuable Pokémon cards to his two grandsons, Abe and Bob. The only direction in the will is to “divide the values of cards as evenly as possible”. As the executor of Mike’s will, you have priced each of the Pokémon cards to obtain an accurate monetary value. You are to decide how to divide up the Pokémon cards into two piles to minimize the difference in the sum of values of each pile.

For example, assume you have the following $n = 8$ Pokémon cards:

Card	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8
Value	2	1	3	1	5	2	3	4

After a lot of work, you figure out that you could divide the cards in the following way:

Abe	c_1, c_3, c_5	10
Bob	c_2, c_4, c_6, c_7, c_8	11

This gives \$10 worth of cards to Abe and \$11 worth of cards to Bob. Is this the best division? Your job is to solve this problem generally for n cards where each card c_i has a positive integer value v_i . Your solution is to compute how the cards should be divided and the value of each pile. The input and output examples are as follows:

2.3.2 基本要求

给出最优划分方案：逐个输入卡牌价值，按‘总价值-选择卡片的下标’输出划分方案

2.3.3 数据结构设计

(1)python 实现

```
# 列表存储时间花费和卡牌价值
cards=[]
DP_time=[]
Brute_time=[]
```

(2)c++ 实现

```
struct card{
    int value;
    int num;
}cards[1000];
```

2.3.4 功能说明（算法思想、函数、类）

0-1 背包：dp[i][j]=1 表示前 i 张卡片能表示 j 值，从 dp[0][0]=1 开始 dp；最后从 sum/2 开始向更小值检测，第一个=1 的 j 为结果；可以改进为原地 dp。

(1) python 实现

```
# dp 实现
def dp(cards):
    n = len(cards)
```

```

total = sum(cards)
target = total // 2

dp= [0]*(target+1)
path = [[] for _ in range(target + 1)]

dp[0] = 1
path[0] = []

for i in range(n):
    for j in range(target, cards[i]-1, -1):
        if dp[j - cards[i]] > 0 and dp[j] == 0:
            dp[j] = 1
            path[j] = path[j - cards[i]] + [i]

for i in range(target, -1, -1):
    if dp[i] > 0:
        return i,path[i]
# 暴力实现
def brute_force(cards):
    n = len(cards)
    ans_total=0
    path=[]

    for i in range((1 << n)-1):
        total = 0
        for j in range(n):
            if (i >> j) & 1:
                total += cards[j]

        if total > sum(cards)//2:
            total=sum(cards) - total

        if total > ans_total:
            ans_total = total
            path = []
            for j in range(n):
                if (i >> j) & 1:
                    path.append(j)

    return ans_total, path

```

(2) cpp 实现

// 贪心不能最优: 6、5、4、3、3, 贪心差值 3, 最优解差值 1

// 01 背包

```
int dpp(int n)
{
    int sum=0;
    for(int i=0;i<n;i++){
        sum+=cards[i].value;
    }

    dp[0]=1;

    // 原地 dp
    for(int i=0;i<n;i++){
        for(int j=sum/2;j>=cards[i].value;j--){
            if(dp[j-cards[i].value])dp[j]=1;
        }
    }

    int ans1=sum;

    for(int j=sum/2;j>=0;j--){
        if(dp[j]) return ans1=j;
    }

    return ans1;
}
```

2.3.5 调试分析（算法分析和输入输出）

原地 Dp 复杂度 $O(\text{sum} * n)$; 暴力复杂度 $O(2^n)$

```
# 测试代码
N=[5,10,15,18,20,21]
cards=[]
DP_time=[]
Brute_time=[]

for n in N:
    cards.clear()
    for i in range(n):
        cards.append(random.randint(1, 100))

    start1 = time.time()
    res1,path1=dp(cards)
    end1 = time.time()
    DP_time.append(end1 - start1)

    start2 = time.time()
```

```

res2,path2=brute_force(cards)
end2 = time.time()
Brute_time.append(end2 - start2)

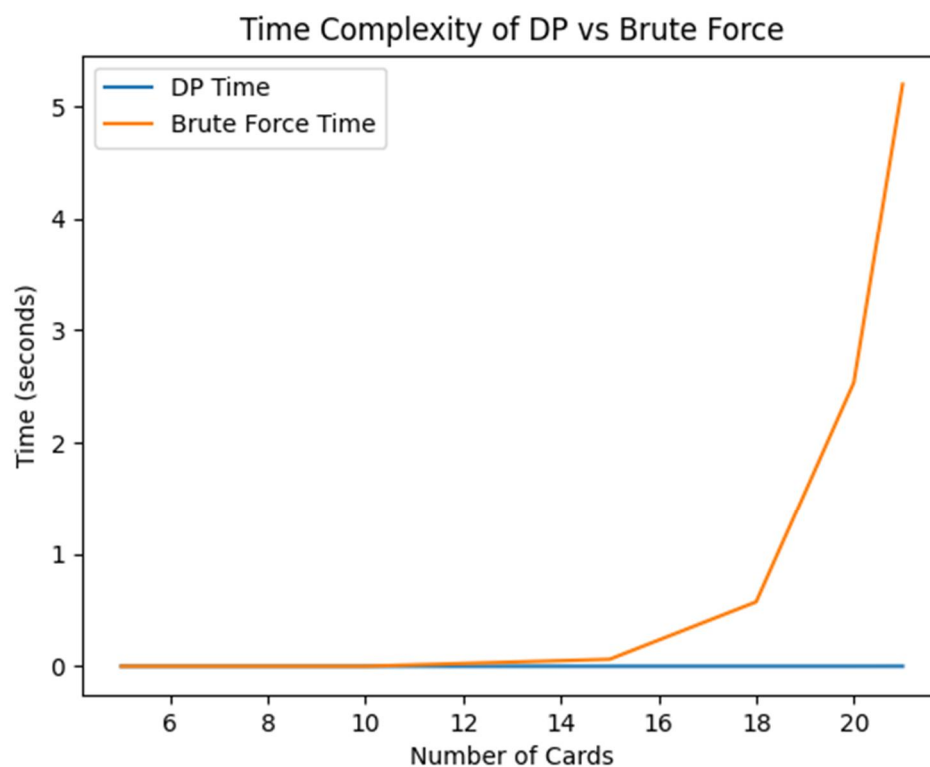
# print("DP Result: ", res1, "Path: ", path1, "Time: ", end1
- start1)
# print("Brute Force Result: ", res2, "Path: ", path2, "Time:
", end2 - start2)

plt.plot(N, DP_time, label='DP Time')
plt.plot(N, Brute_time, label='Brute Force Time')
plt.xlabel('Number of Cards')
plt.ylabel('Time (seconds)')
plt.title('Time Complexity of DP vs Brute Force')
plt.legend()
plt.show()

plt.savefig('time_complexity.png')
plt.clf()

```

运行结果:



3. 实验总结

本次期中大作业围绕排序算法、分治算法和动态规划三大核心算法展开，通过理论分析、代码实现和实验对比，深入探讨了不同算法在不同场景下的性能表现和适用性。以下是本次实验的主要收获和总结：

1. 题目一：排序算法综合应用

核心目标：对考生成绩按总分、单科成绩进行多级排序，并处理不同规模数据（小规模内存排序 vs 大规模 Top-K 查询）。

亲身体会：快速排序在 $n \leq 50,000$ 时表现优异 ($O(n \log n)$)，远优于冒泡排序 ($O(n^2)$)；小根堆 ($O(n \log m)$) 高效解决 Top-K 问题，适用于 n 极大时（如 $m = 1000$, $n = 1e6$ ）。

优化思考：若数据分布已知（如分数正态分布），可尝试桶排序进一步加速；对于极大规模数据（如分布式环境），可结合外部排序（归并排序+分块处理）。

2. 题目二：分治算法（第 K 小元素选择）

核心目标：对比三种算法（堆选择、随机划分、中位数划分）在不同 n 和 k 下的性能。

亲身体会：堆选择：适合 $k \ll n$ （如 $k = 10$, $n = 1e6$ ），但 k 接近 $n/2$ 时退化至 $O(n \log n)$ ；随机划分：平均 $O(n)$ ，但最坏 $O(n^2)$ （如极端划分情况）；中位数划分：稳定 $O(n)$ ，但常数因子较大，小规模数据反而不如随机划分。

优化思考：实际应用中可混合策略：小规模用随机划分，大规模用中位数划分；若数据允许预处理，构建静态结构（如线段树）可支持多次查询。

3. 题目三：动态规划（卡牌划分问题）

核心目标：将卡牌划分为两堆，使价值和差值最小（0-1 背包变种）。

亲身体会：动态规划 ($O(n \cdot \text{sum})$) 在 $n \leq 20$ 时显著优于暴力枚举 ($O(2^n)$)；原地 DP 节省空间，但需注意 sum 较大时的内存限制（如 $\text{sum} > 1e6$ ）。

优化思考：若 sum 极大，可尝试折半枚举+双指针或随机化算法（如模拟退火）；另外，贪心算法虽快，但无法保证最优解（如测试用例 $[6, 5, 4, 3, 3]$ ）。

本次实验不仅巩固了经典算法的理论基础，更通过实践加深了对时间复杂度、空间复杂度权衡的理解，为后续算法设计与优化提供了宝贵经验。